Artificial Neural Networks

Assignment II - Regression with Neural Networks

1st Leonardo Henriques
University of Madeira, n°2120922

2nd Leonardo Sousa
University of Madeira, n°2123822

***Abstract*—This study evaluates the performance of shallow neural networks in addressing two distinct environmental regression tasks: short-term wind speed forecasting and apparent temperature inference. A rigorous methodology was implemented, focusing on specialized pre-processing techniques such as trigonometric decomposition for cyclic features and logarithmic scaling for skewed distributions. Through a systematic grid search across various hidden layer sizes and activation functions, optimal architectures were identified and validated through 30 independent trials per configuration. The final models achieved high predictive precision, yielding a Mean Absolute Error (MAE) of 0.9669 m/s for wind speed and 0.0432 ºC for apparent temperature. These results demonstrate that the synergy between robust feature engineering and hyperparameter optimization enables shallow networks to achieve near-formulaic accuracy in complex meteorological forecasting problems.**

I. Introduction

This report was developed within the scope of the *Artificial Neural Networks* course. It details the training and evaluation of neural network models for two distinct regression tasks. For the *windSpeed.csv* dataset, the objective is to predict wind speed one minute into the future based on historical data. Conversely, for the *weatherHistory.csv* dataset, the goal is to infer the apparent temperature from various weather measurements.

II. Methodology: *windSpeed dataset*

A. Data Characterization

The initial phase of this study involves exploratory data analysis and preprocessing to build a reliable predictive model for wind speed at Madeira International Airport. According to the assignment guidelines, the dataset comprises the following variables:

- **Timestamp**: Date and time of the recording (yyyy-mm-dd hh:mm:ss format);
- **Wind Direction**: Wind direction (in degrees (º));
- **Wind Speed**: Wind velocity (in m/s);

B. Data Cleaning and Pre-processing

Upon loading the dataset, 1,048,575 observations were identified. The raw data contained two primary columns: *CREATEDATE*, which aggregated the timestamp and wind direction, and *DATA*, which recorded the wind speed. Initial preprocessing involved decomposing the *CREATEDATE* column into distinct timestamp and direction variables, followed by renaming the features for clarity.

Subsequently, the dataset was inspected for inconsistencies, with a strong emphasis on temporal continuity, a critical requirement for time-series forecasting. While no duplicate records were found, minor missing values were detected (two for wind speed and one for wind direction). Given the brief duration of these gaps (a maximum of seven seconds, excluding a massive gap in February, as illustrated in Figure 1), linear interpolation was applied. Finally, the dataset was segmented into continuous blocks wherever chronological interruptions exceeded 70 seconds. This strategy divided the data into two primary contiguous blocks, effectively isolating the large data gap observed in February.

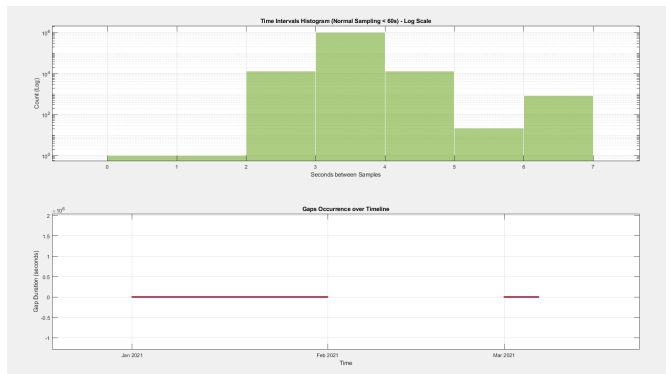


Figure 1: Time intervals and missing gaps in the windSpeed dataset.

C. Feature Engineering and Selection

To enhance model performance and capture underlying temporal and physical dynamics, the following features were engineered:

- **Cyclic Transformation of Wind Direction:** Sine and cosine functions were applied to the wind direction to preserve its circular nature. This geometric encoding prevents the neural network from incorrectly interpreting the transition between 359º and 1º as a massive 358º numerical difference.

Listing 1: Trigonometric encoding of the wind direction

```
theta_rad = deg2rad(data.WindDirection);
data.Dir_Sin = sin(theta_rad);
data.Dir_Cos = cos(theta_rad);
```

- **Temporal Encoding:** The time of day was converted into a continuous decimal format and subsequently transformed into periodic sine and cosine components. This ensures the model recognizes the cyclic proximity of late-night and early-morning hours.

Listing 2: Trigonometric encoding of the timestamp

```
horas_decimais = hour(data.Time) + minute(data.Time)/60
      + second(data.Time)/3600;
data.Hour_Sin = sin(2 * pi * horas_decimais / 24);
data.Hour_Cos = cos(2 * pi * horas_decimais / 24);
```

- **Wind Acceleration:** Derived using the finite difference between the current and previous wind speed measurements. Exception handling was implemented at the beginning of each block to avoid invalid cross-block calculations.

Listing 3: Wind Acceleration

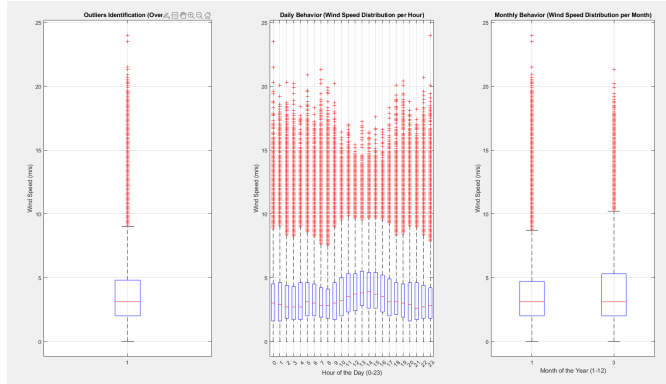
```
data.Wind_Acc = [0; diff(data.WindSpeed)];
idx_block_starts = [1; find(diff(data.BlockID) > 0) +
     1];
data.Wind_Acc(idx_block_starts) = 0;
```

Finally, the target variable (*Target_WindSpeed*) was constructed. For each observation t, a look-ahead search was performed to identify the first future wind speed recorded at least 60 seconds later.

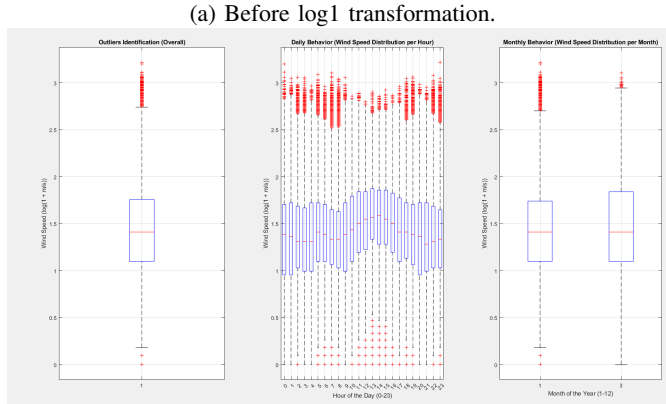
Listing 4: Look-ahead loop to find the future values of wind speed

```
time_numeric = cumsum(data.TimeDiff);
speeds = data.WindSpeed;
targets = NaN(size(speeds));
block_ids = data.BlockID;

block_starts = [1; find(diff(block_ids) > 0) + 1];
block_ends = [block_starts(2:end) - 1; length(block_ids)];

for b = 1:length(block_starts)
    start_idx = block_starts(b);
    end_idx = block_ends(b);
    if end_idx > start_idx
        j = start_idx;
        for i = start_idx:end_idx
            while j <= end_idx && (time_numeric(j) -
     time_numeric(i)) < 60
                j = j + 1;
            end
            if j <= end_idx
                targets(i) = speeds(j);
            end
        end
    end
end
```

We also applied a logarithmic transformation to the wind speed data, as it exhibited a right-skewed distribution. This allowed us to significantly reduce the number of outliers and stabilize model convergence, as the models were less affected by extreme wind speed values, leading to a smoother optimization process.



(a) Before log1 transformation.



(b) After log1 transformation.

Figure 2: Comparison of wind speed boxplots before and after log1 transformation.

To further visualize the impact of these pre-processing steps, Figure 3 illustrates the temporal behavior of the wind speed (after logarithmic scaling), the wind direction, and the engineered wind acceleration.

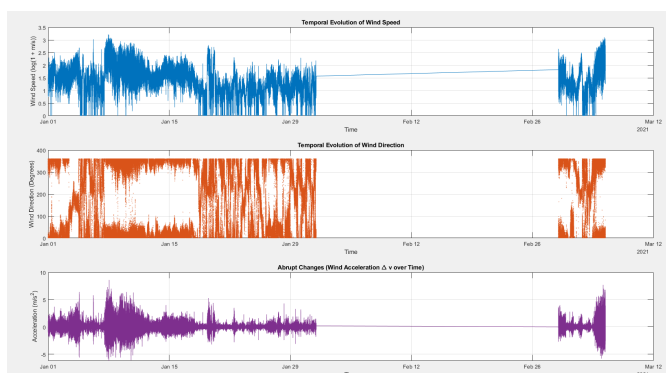
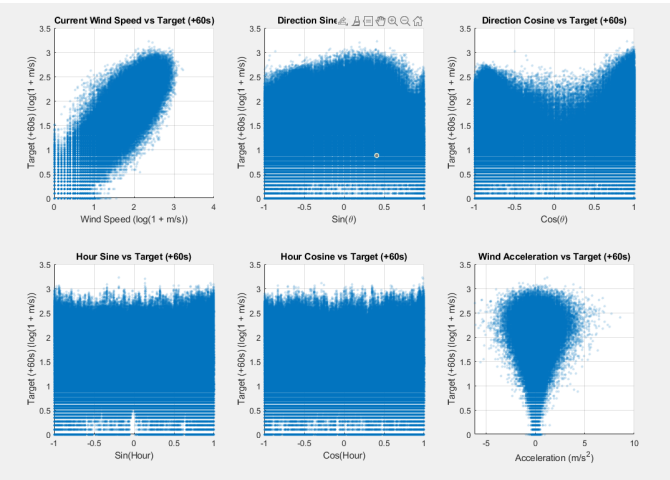


Figure 3: Temporal evolution of the log-transformed wind speed, wind direction, and acceleration.

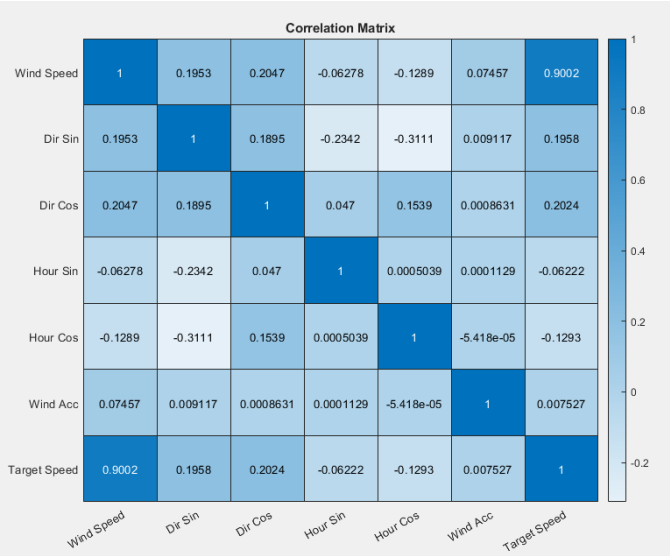
The temporal evolution of the processed features confirms the effectiveness of the transformations. The top panel shows the wind speed after the $\log(1+v)$ scaling, which compresses the range of values and stabilizes the variance, making the series more suitable for gradient-based optimization. The large

data gap in February is clearly visible, justifying the block-based segmentation strategy. The bottom panel displays the wind acceleration, which captures the instantaneous rate of change and provides the network with a representation of the physical momentum, a key factor in improving short-term predictive accuracy.

The final curated dataset comprises the original inputs, the newly engineered features, and the target variable. Following this phase, a final correlation analysis was conducted.



(a) Scatter plots.



(b) Correlation matrix.

Figure 4: Feature analysis.

As shown in Figure 4, the engineered features exhibit low linear correlation with the target variable, with the exception of the current wind speed, which is expected given the target is a temporal displacement of this feature. However, this lack of linear correlation does not hinder the predictive capability of the model, as neural networks excel at extracting complex, non-linear relationships from such multi-dimensional data.

Based on the exploratory data analysis and the feature engineering process, the final subset of input features selected to train the predictive model consists of the following six variables:

- **WindSpeed** (log-transformed): The current wind velocity, acting as the primary baseline for the short-term forecast.
- **Dir_Sin and Dir_Cos**: The decomposed trigonometric components of the wind direction, providing a continuous spatial orientation without the 360º-to-0º discontinuity.
- **Hour_Sin and Hour_Cos**: The cyclical encoding of the timestamp, enabling the network to learn diurnal patterns driven by solar heating.
- **Wind_Acc**: The instantaneous rate of change in wind speed, informing the model whether the wind momentum is currently in an upward or downward trend.

Original variables such as the raw timestamp, block identifiers, and linear wind direction were preserved in the dataset but excluded from the model training process. This prevents redundancy and ensures the neural network does not learn spurious artifacts or spatial discontinuities.

This compact and highly informative feature set aligns with best practices for shallow neural network training, ensuring a balanced, scaled, and physically sound input space capable of capturing both the momentum and the cyclical patterns of the wind.

III. Methodology: *weatherHistory dataset*

A. Data Characterization

The data set consists of 96,453 values and 8 features. Our objective is to analyze them and, through data cleaning and feature engineering, obtain a data set to train a neural network capable of predicting the apparent temperature, it being our variable target.

The features are as follows:

- **Temperature_C**: The temperature at a given moment in Celsius (Cº);
- **Apparent_Temperature_C**: The equivalent perceived by humans, also in Celsius (Cº);
- **Humidity**: The humidity percentage of said moment, its values are presented between 0 and 1;
- **Wind_Speed_km_h_**: The wind's speed in kilometers per hour (km/h);
- **WindBearing_degrees_**: The direction of which the wind is blowing from in degrees;
- **Visibility_km_**: The maximum distance that an object can be perceived in the current weather in kilometers (km);
- **Cloud_Cover**: If there are clouds covering the sky or not, with one representing a fully clouded sky and zero representing clear skies;
- **Pressure_millibars_**: The atmospheric pressure measured in millibars (mbar).

It is important to note that in the data set the feature Cloud Cover is presented as *LoudCover*, as this is most likely a error,

and as I found no use for the feature and later removed it. I decided to not change it's name in the data set and rename it in the report.

B. Data Cleaning and Pre-processing

Firstly we analysed the amount of zeros and values missing. There were no values missing in the data set. Although there were zeros in all the features.

After analysing the distribution of the data, the box plots, the correlation matrix and scatterplot we can notice that our data set has errors in these last analysis. There is a small amount of zeros in the distribution of the feature related to the atmospheric pressure and humidity. Which can't be possible in natural environments, which means that they are more likely to be errors. Furthermore the feature Cloud Coverage possesses mostly zero values Which explains why they appear as a *NaN* in the Correlation Matrix.

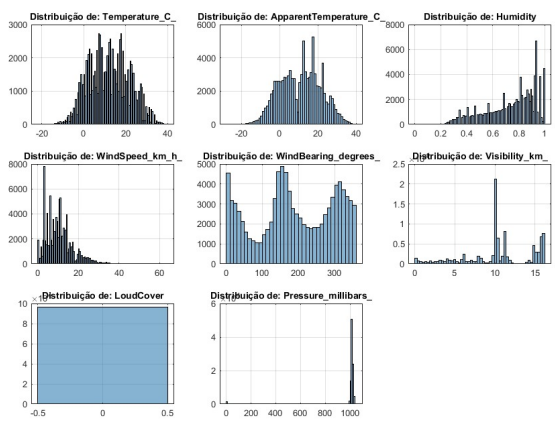


Figure 5: Distribution of the Features.

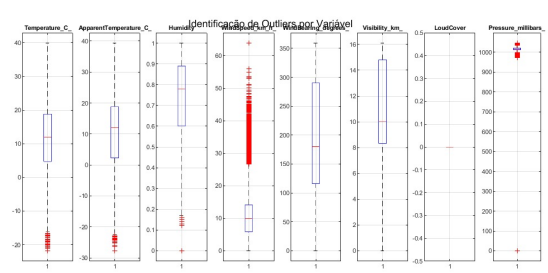


Figure 6: Boxplot of the Features.

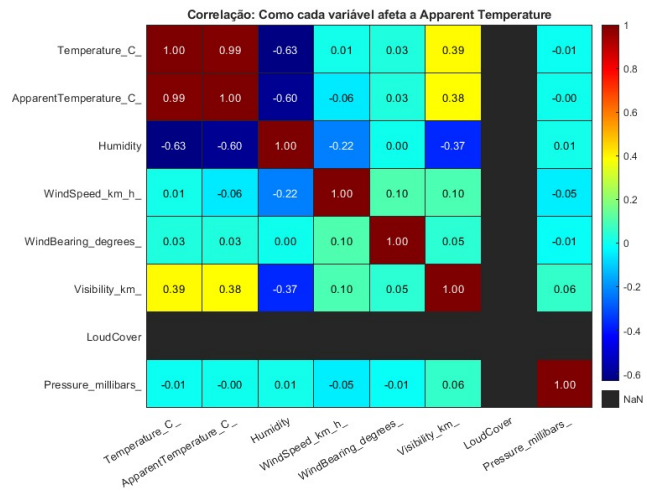


Figure 7: Correlation Matrix of the features.

As observed Correlation Matrix we can tell that Temperature and Humidity have a very High correlation to our target variable, *Apparent_Temperature_C_*, Visibility also has some correlation to it too.

Given all this information I decided to clean the zeros in Humidity and Pressure and also remove the feature Cloud Cover.

To clean the data I decided to use Linear Interpolation to correct the zeros and deleted any missing value the interpolation didn't manage to correct.

After a subsequent analysis, it was found that the distribution of the feature *WindSpeed* possesses too many outliers that could affect the quality of the neural network. Unfortunately, this was found after finding the most optimal hyperparameters for the neural network. As such, we lacked the time to clean or retrain the neural network.

C. Feature Engineering and Selection

For the feature engineering I decided to think about what could possibly affect the temperature. Additionally in order to try and make Wind Direction possibly relevant, and to help with the neural network training, I decided to remove it and decompose it in both Sine and Cosine, although I still had doubts it would provide something useful. With this in mind I created:

- **Wind_Sin and Wind_Cos**: The decomposition of the feature Wind Direction in degrees;

Listing 5: Wind Direction Decomposition

```
data.Wind_Sin=sin(deg2rad(data.WindBearing_degrees_));
data.Wind_Cos=cos(deg2rad(data.WindBearing_degrees_));
```

- **Heat_Index_Proxy**: Obtained after multiplying Temperature and Humidity. It represents a simpler substitution of the more complex calculations in Heat Index;

Listing 6: Heat_Index_Proxy

```
data.Heat_Index_Proxy = data.Temperature_C_ .* data.
    Humidity;
```

- **Wind_Chill_Effect**: By multiplying Wind Speed and Temperature we should have something that represents the wind's effect in the temperature.

Listing 7: Wind_Chill_Effect

```
data.Wind_Chill_Effect = data.WindSpeed_km_h_ .* data.
    Temperature_C_;
```

- **Hum_Press_Ratio**: We get it by multiplying Humidity with Pressure, it represents the density and stability of air, since an higher pressure and humidity means a higher retention of heat.

Listing 8: Hum_Press_Ratio

```
data.Hum_Press_Ratio = data.Humidity .* data.
    Pressure_millibars_;
```

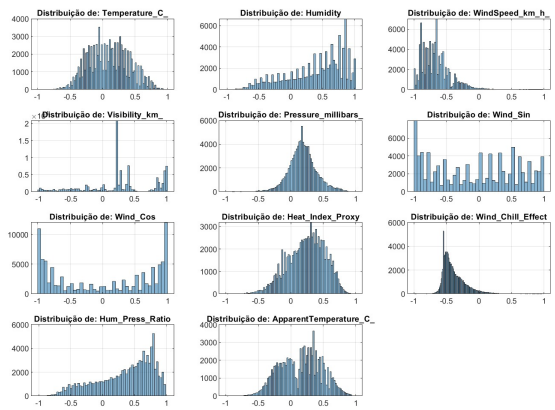


Figure 8: Distribution of the Features after Feature Engineering.

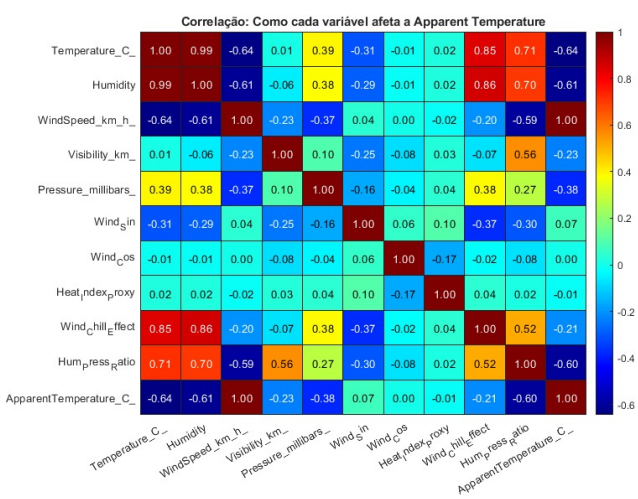


Figure 9: Correlation Matrix after feature engineering.

After analysing with the same graphs previously mentioned I arrived at the conclusion that the correlation between the target variable and the decomposition of Wind Direction does not justify their creation and as such I deleted them. Ending with the following Features:

- **Temperature_C_**: Current air temperature;
- **Apparent_Temperature_C_**: Target variable (perceived temperature);
- **Humidity**: Relative humidity;
- **WindSpeed_km_**: Wind velocity;
- **Visibility_km_**: Visual distance;
- **Pressure_millibars_**: Atmospheric pressure;
- **Heat_Index_Proxy**: Combined temperature and humidity;
- **Wind_Chill_Effect**: Combined wind and temperature;
- **Hum_Press_Ratio**: Combined humidity and pressure.

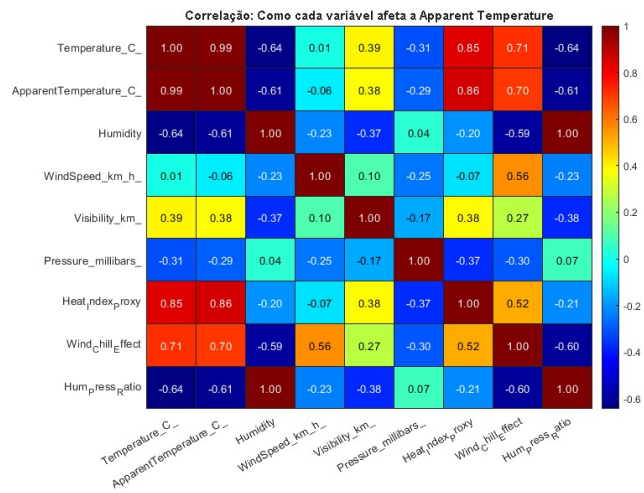


Figure 10: Correlation Matrix of the Final Features.

D. Data Partitioning

Since the weather dataset observations represent independent atmospheric measurements rather than a sequential forecasting problem, a randomized data partitioning strategy was adopted. The dataset was randomly shuffled and split into 90% for training and validation (86,807 samples), and 10% for the final held-out test set (9,646 samples). This randomized approach ensures that both sets contain a representative distribution of various weather conditions, mitigating any seasonal or chronological bias present in the raw data order.

IV. Models Training

A. Network Architecture and Hyperparameters

For both regression tasks, a shallow feedforward neural network architecture, specifically MATLAB's `fitnet`, was employed with a single hidden layer. While forecasting wind speed is inherently a dynamic time-series problem—typically requiring recurrent architectures such as NARNet or NARXNet—our data preparation strategy allowed for a simpler and more efficient approach. By explicitly engineering the target variable (locating the wind speed exactly one minute into the future and aligning it with the current inputs), the

temporal shift was inherently embedded into the dataset. This transformation effectively reduced the dynamic forecasting task to a standard static multivariate regression problem, fully justifying the use of a standard feedforward architecture for both the wind speed and the weather station datasets. To determine the optimal network configuration, a grid search was conducted over two primary hyperparameters: the number of hidden neurons and the activation function. The hidden layer size was varied across four configurations (5, 10, 15, and 20 neurons) to evaluate the optimal balance between model capacity and generalization, aiming to prevent both underfitting and overfitting. Regarding the hidden layer's activation function, two distinct mathematical approaches were evaluated: the Hyperbolic Tangent Sigmoid (`tansig`) and the Positive Linear (`poslin`, equivalent to ReLU). The `tansig` function was selected because its smooth, zero-centered nature (outputting values between -1 and 1) is traditionally highly effective for capturing complex, non-linear relationships in normalized regression tasks. Conversely, the `poslin` function was introduced to assess its sparsity and non-saturating properties for positive values, which helps mitigate the vanishing gradient problem and is particularly well-suited for unbounded physical measurements.

B. Training Strategy and Approach

To ensure robust and statistically significant results, the training strategy was designed in accordance with the Central Limit Theorem. The hyperparameter grid search yielded 8 distinct configuration points, representing all combinations of the selected neuron counts and activation functions. For each configuration, the neural network was initialized and trained 30 independent times. This comprehensive evaluation, totaling 240 training runs per dataset, allowed for a rigorous assessment of model convergence and stability, mitigating the variance inherent to stochastic weight initialization. During the optimization phase, models for both datasets were trained using the Mean Squared Error (MSE) as the primary performance function. MSE was chosen to effectively penalize larger prediction errors during the weight update process, which was driven by the Levenberg-Marquardt backpropagation algorithm (`trainlm`). Once the optimal combination of hyperparameters was identified, based on the lowest average validation MSE, a final, definitive model was trained. To provide a highly interpretable assessment of the model's real-world accuracy, this final network was evaluated against an isolated, completely unseen test dataset using the Mean Absolute Error (MAE) metric. This final evaluation ensures that the reported performance reflects the absolute physical deviation in the target variables' native units.

V. Results

A. Performance and Metrics (windSpeed dataset)

The empirical results of the hyperparameter grid search for the wind speed forecasting task are summarized in Table I. Following the training of 30 independent models for each of the 8 configurations, the `poslin` (ReLU) activation function consistently demonstrated superior performance compared to `tansig`, particularly as the network complexity increased.

Table I: Neural Network Optimization Results for windSpeed dataset

Neurons	Activation	Mean Val MSE	Variance Val MSE
5	"tansig"	0.041729	3.2583×10^{-8}
5	"poslin"	0.041814	1.4747×10^{-7}
10	"tansig"	0.041574	4.4508×10^{-8}
10	"poslin"	0.041441	4.1427×10^{-8}
15	"tansig"	0.041476	3.7216×10^{-8}
15	**"poslin"**	**0.041365**	$\mathbf{3.2485 \times 10^{-8}}$
20	"tansig"	0.041599	2.9701×10^{-8}
20	"poslin"	0.041365	1.8877×10^{-8}

Note: Highlighted row indicates the optimal configuration based on the trade-off between error and model parsimony.

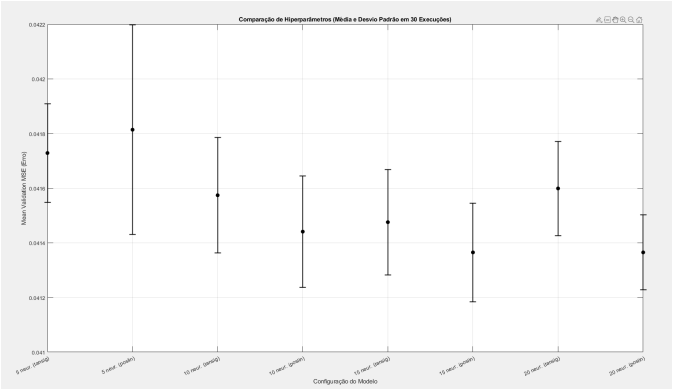


Figure 11: Comparative analysis of mean and standard deviation of MSE across different configurations.

As illustrated in Figure 11, the `poslin` models generally achieved lower error rates and exhibited greater stability across different neuron counts. The configuration with 15 neurons and the `poslin` activation function was selected as the optimal architecture. While the 20-neuron `poslin` model yielded a similar mean error, the 15-neuron variant offers a more efficient balance between predictive power and computational simplicity, effectively mitigating the risk of overfitting.

Following the selection of the optimal configuration, a final definitive model was trained on the full training set and validated. The optimization curve, depicted in Figure 12, demonstrates a smooth and consistent convergence of both training and validation errors, indicating a robust learning process and high generalization potential.

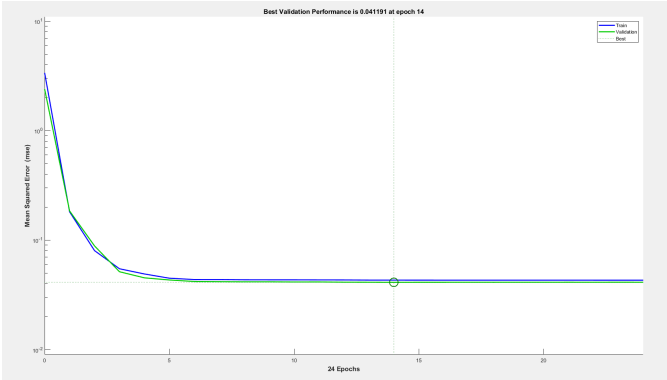


Figure 12: Optimization curve showing the progression of training and validation errors.

The final assessment of the model's performance was conducted using a held-out test set comprising 104,854 independent samples. The resulting Mean Absolute Error (MAE) was 0.9669 m/s. This value is highly significant, as it represents an average prediction error of less than 1 m/s in future wind speed, confirming the model's reliability for short-term meteorological forecasting.

B. Performance and Metrics (weatherHistory dataset)

The hyperparameter optimization results for the weather station dataset, involving the prediction of apparent temperature, are presented in Table II. Similar to the previous task, the grid search evaluated the impact of neuron count and activation functions over 30 independent trials per configuration.

Table II: Neural Network Optimization Results for weatherHistory dataset

Neurons	Activation	Mean Val MSE	Variance Val MSE
5	"tansig"	0.081584	4.7407×10^{-4}
5	"poslin"	0.086966	1.2022×10^{-3}
10	"tansig"	0.033847	3.8684×10^{-5}
10	"poslin"	0.022061	1.8038×10^{-4}
15	"tansig"	0.021651	1.8338×10^{-5}
15	"poslin"	0.018023	1.8953×10^{-4}
20	"tansig"	0.020038	3.4357×10^{-5}
20	**"poslin"**	**0.011805**	$\mathbf{9.4712 \times 10^{-5}}$

Note: Highlighted row indicates the optimal configuration (20 neurons, `poslin`).

The empirical data demonstrates a clear correlation between network capacity and predictive accuracy. For this specific task, the `poslin` activation function coupled with 20 hidden neurons achieved the most significant error reduction, yielding a mean validation MSE of 0.011805. This trend, visualized in Figure 13, suggests that the additional neurons allowed the network to better map the complex interactions between temperature, humidity, and pressure that define the apparent temperature.

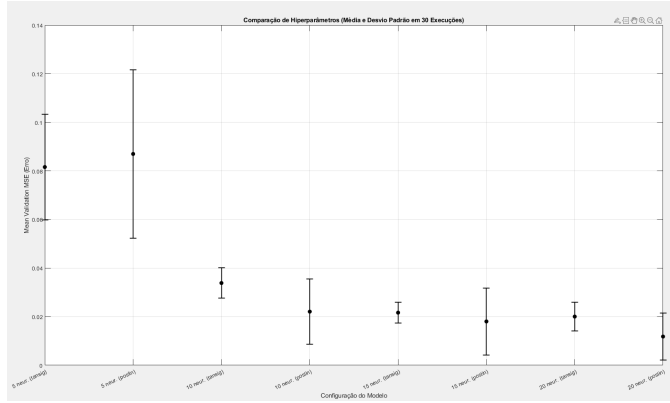


Figure 13: Performance comparison of activation functions and hidden layer sizes for the weatherHistory dataset.

Following the identification of the optimal configuration, a final model was trained. The convergence behavior, shown in Figure 14, reflects a stable optimization process with errors stabilizing at very low values.

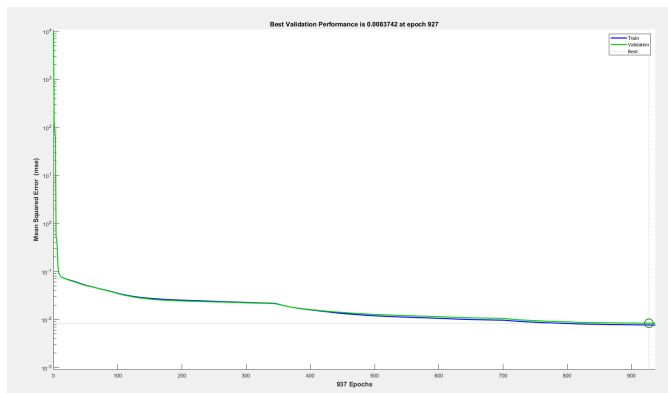


Figure 14: Training and validation error progression for the optimal weatherHistory model.

The final evaluation on the isolated test set yielded a Mean Absolute Error (MAE) of 0.0432. This suggests that the engineered features, particularly the interaction between temperature and humidity, provided the network with sufficient information to reconstruct the apparent temperature formula with near-perfect accuracy.

VI. Discussion

The results obtained across both regression tasks highlight the critical role of data preparation and feature engineering in the training of shallow neural networks. For the *windSpeed* dataset, the challenge was primarily temporal and stochastic. By transforming the wind direction and timestamp into cyclic components and applying a logarithmic scaling to the wind speed, we were able to provide the network with a more stable and physically consistent input space. The final MAE of 0.9669 m/s is a significant achievement for a single-minute forecast using a standard feedforward architecture,

as it demonstrates that the network effectively captured the momentum and short-term trends of the wind.

In contrast, the *weatherHistory* dataset presented a more deterministic problem. Apparent temperature is physically derived from current weather measurements, such as humidity and real temperature. This was reflected in the remarkably low MAE of 0.0432. The high precision achieved here confirms that the engineered features, such as the heat index proxy and the wind chill effect, successfully exposed the underlying physical formulas to the network.

Across both datasets, the `poslin` (ReLU) activation function proved to be more efficient than the traditional `tansig`. This can be attributed to the fact that ReLU does not suffer from saturation for large positive values, which was particularly beneficial for handling the wide ranges of temperature and wind velocity. Furthermore, the low variance observed across the 30 independent training runs for each configuration validates the statistical significance of our hyperparameter selection and confirms that the models converged to stable, reproducible solutions.

VII. Conclusion

This study successfully demonstrated the implementation of neural network-based regression for two distinct environmental forecasting problems. Through a rigorous methodology encompassing data cleaning, anomaly analysis, and specialized feature engineering, we transformed raw, sometimes inconsistent measurements into highly informative training sets.

The systematic hyperparameter grid search proved essential in identifying the optimal balance between network capacity and generalization. While shallow architectures were employed, the results show that when combined with robust preprocessing, these models are capable of achieving high predictive accuracy. Specifically, the wind speed forecast achieved an average error of less than 1 m/s, and the apparent temperature inference achieved near-formulaic precision.

In summary, the project confirms that the success of a neural network model is fundamentally tied to the precision of the data preparation and the clarity of its representation. The techniques refined throughout this study—most notably the trigonometric encoding of periodic variables and the strategic selection of non-linear transfer functions—establish a robust framework for addressing more complex meteorological forecasting and environmental regression challenges in the future.

APPENDIX

Listing 9: windspeed_data_cleaning.m

```
%% Projeto 2: Redes Neuronais Artificiais - Wind Speed
% Script: Carregamento, Limpeza, Análise de Gaps e Segmentação por Blocos
clc; clearvars; close all;

%% 1. Importação e Exploração Inicial (Para Relatório)
path = "windSpeed.csv";

% Verificar se o ficheiro existe antes de carregar
if ~isfile(path)
    error('Ficheiro não encontrado no caminho: %s', path);
end

% 'preserve' mantém os nomes originais; 'detectImportOptions' ajuda na robustez
opts = detectImportOptions(path);
opts.VariableNamingRule = 'preserve';
data = readtable(path, opts);

% Atribuir nomes amigáveis para manipulação interna
data.Properties.VariableNames{1} = 'RawTimestamp';
data.Properties.VariableNames{2} = 'WindSpeed';

fprintf('==============================================\n');
fprintf('        1. EXPLORAÇÃO E TRATAMENTO        \n');
fprintf('==============================================\n');
fprintf('Observações: %d | Variáveis Originais: %d\n', size(data,1), size(data,2));
fprintf('----------------------------------------------\n');

%% 2. Processamento de Strings e Separação de Variáveis
fprintf('Processando Timestamps e Direção do Vento...\n');

% 1. Converter para string e remover aspas ou espaços nas extremidades
rawStrings = strtrim(string(data.RawTimestamp));
rawStrings = strrep(rawStrings, '"', '');

% 2. Extração da Data (os primeiros 19 caracteres: yyyy-mm-dd hh:mm:ss)
datesStr = extractBefore(rawStrings, 20);

% 3. Extração da Direção (procurar a última sequência de dígitos na string)
dirMatches = regexp(rawStrings, '\D+(\d+)$', 'tokens', 'once');

% Inicializar vetor de direções
directions = NaN(size(rawStrings));

% Converter células extraídas para números
validDir = ~cellfun(@isempty, dirMatches);
if any(validDir)
    tmp_dir = cellfun(@(x) str2double(x{1}), dirMatches(validDir));
    directions(validDir) = tmp_dir;
end

% 4. Conversão final e limpeza da tabela
data.Time = datetime(datesStr, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
data.WindDirection = directions;
data.RawTimestamp = []; % Remover coluna bruta

% Reordenar para melhor visualização
data = data(:, {'Time', 'WindSpeed', 'WindDirection'});

%% 2.5 Verificação de Registos Duplicados
fprintf('\n--- Verificação de Registos Duplicados ---\n');
[~, uniqueIdx] = unique(data.Time, 'first');
num_duplicados = height(data) - length(uniqueIdx);
if num_duplicados > 0
    data = data(uniqueIdx, :);
    fprintf('Removidas %d observações duplicadas (mesmo timestamp).\n', num_duplicados);
else
    fprintf('Nenhuma observação duplicada encontrada.\n');
end

%% 3. Tratamento de Valores em Falta (NaN)
fprintf('\n--- Verificação de Qualidade Pós-Separação ---\n');

nanSpeed = sum(isnan(data.WindSpeed));
nanDir = sum(isnan(data.WindDirection));

fprintf('Valores em falta na Velocidade: %d\n', nanSpeed);
fprintf('Valores em falta na Direção:    %d\n', nanDir);

% Preenchimento com Interpolação Linear
```

```
if nanSpeed > 0
    data.WindSpeed = fillmissing(data.WindSpeed, 'linear');
end
if nanDir > 0
    data.WindDirection = fillmissing(data.WindDirection, 'linear');
end
fprintf('---------------------------------------------\n');

%% 4. Segmentação em Blocos (Tratamento de Sequências Disconexas)
% Definimos que qualquer gap > 70 segundos quebra a sequência lógica
thresholdRuptura = 70;
data.TimeDiff = [0; seconds(diff(data.Time))];

% Criar ID de Bloco: aumenta sempre que há uma ruptura
data.BlockID = cumsum(data.TimeDiff > thresholdRuptura) + 1;

numBlocos = max(data.BlockID);
fprintf('\n=============================================\n');
fprintf('     4. SEGMENTAÇÃO POR BLOCOS (GAPS)     \n');
fprintf('=============================================\n');
fprintf('Threshold de Ruptura: %d segundos\n', thresholdRuptura);
fprintf('Número de blocos independentes encontrados: %d\n', numBlocos);
fprintf('---------------------------------------------\n');

%% 5. Feature Engineering (Decomposição Vetorial Cíclica)
fprintf('\n=============================================\n');
fprintf('          5. FEATURE ENGINEERING          \n');
fprintf('=============================================\n');
fprintf('Decompondo a Direção e Velocidade em vetores contínuos (U e V)...\n');

% Converter graus para radianos
theta_rad = deg2rad(data.WindDirection);

% 1. Decomposição da Direção (Bússola)
data.Dir_Sin = sin(theta_rad);
data.Dir_Cos = cos(theta_rad);

% 2. Ciclo Diurno (Hora do Dia Cíclica)
% O vento é movido a aquecimento solar. Um relógio de 24h cíclico ajuda o modelo a distinguir o dia da noite.
horas_decimais = hour(data.Time) + minute(data.Time)/60 + second(data.Time)/3600;
data.Hour_Sin = sin(2 * pi * horas_decimais / 24);
data.Hour_Cos = cos(2 * pi * horas_decimais / 24);

% 3. Momento / Aceleração do Vento (Derivada)
% Indica se o vento está num pico ascendente ou a perder força no instante t
data.Wind_Acc = [0; diff(data.WindSpeed)];
% Anular a aceleração no primeiro instante de cada novo bloco (pois ocorreu um gap temporal antes)
idx_block_starts = [1; find(diff(data.BlockID) > 0) + 1];
data.Wind_Acc(idx_block_starts) = 0;

fprintf('Features Dir_Sin e Dir_Cos criadas com sucesso para evitar saltos de 359  -> 1  !\n');
fprintf('Features Hour_Sin, Hour_Cos e Wind_Acc (Aceleração) adicionadas com sucesso!\n');
fprintf('---------------------------------------------\n');

%% 6. Criação da Variável Alvo (Target) - 1 Minuto no Futuro
fprintf('\n=============================================\n');
fprintf('      6. CRIAÇÃO DA VARIÁVEL ALVO (TARGET)  \n');
fprintf('=============================================\n');
fprintf('Criando target para previsão de WindSpeed 60 segundos no futuro...\n');

% OTIMIZAÇÃO: Usar arrays nativos em vez de aceder à tabela linha a linha
time_numeric = cumsum(data.TimeDiff);
speeds = data.WindSpeed;
targets = NaN(size(speeds));
block_ids = data.BlockID;

% Pré-calcular índices de início e fim de cada bloco (Evita find() O(N) dentro do ciclo)
block_starts = [1; find(diff(block_ids) > 0) + 1];
block_ends = [block_starts(2:end) - 1; length(block_ids)];

for b = 1:length(block_starts)
    start_idx = block_starts(b);
    end_idx = block_ends(b);

    % Só faz sentido calcular se o bloco tiver tamanho suficiente (>1 elemento)
    if end_idx > start_idx
        j = start_idx;
        for i = start_idx:end_idx
            % Avançar o ponteiro j usando matemática rápida (doubles)
            while j <= end_idx && (time_numeric(j) - time_numeric(i)) < 60
                j = j + 1;
            end

```

```
            % Se encontrou uma amostra no futuro que obedeça à regra
            if j <= end_idx
                targets(i) = speeds(j);
            end
        end
    end
end

% Alocar os resultados finais de volta à tabela de uma só vez
data.Target_WindSpeed = targets;

fprintf('Cálculo da feature Target_WindSpeed concluído! Iniciando limpeza de blocos órfãos...\n');

% Limpeza das linhas sem Target (finais de blocos sem futuro conhecido)
linhas_antes = height(data);
data = data(~isnan(data.Target_WindSpeed), :);
linhas_depois = height(data);

fprintf('Target_WindSpeed gerado com sucesso usando pesquisa look-ahead.\n');
fprintf('Linhas removidas (sem 1 min de futuro conhecido): %d\n', linhas_antes - linhas_depois);
fprintf('Dataset final pronto com %d observações.\n', linhas_depois);
fprintf('---------------------------------------------\n');

%% 6.5 Transformação Logarítmica (Preparação para o Modelo)
fprintf('\n--- Transformação Logarítmica (Right Skew) ---\n');
fprintf('Aplicando log1p() a WindSpeed e Target_WindSpeed...\n');
data.WindSpeed = log1p(data.WindSpeed);
data.Target_WindSpeed = log1p(data.Target_WindSpeed);
fprintf('---------------------------------------------\n');

%% 7. Resumo dos Blocos e Exportação
% Identificar os maiores blocos (mais úteis para treino)
stats = groupsummary(data, 'BlockID');
stats = sortrows(stats, 'GroupCount', 'descend');

fprintf('\nResumo dos maiores blocos (Top 5):\n');
disp(head(stats, 5));

fprintf('\nConclusão para o modelo:\n');
fprintf('-> O gráfico de timeline identifica visualmente as rupturas.\n');
fprintf('-> Para treinar a RNA, use janelas de dados dentro do mesmo BlocoID.\n');
fprintf('=============================================\n');

% Drop das features originais redundantes (apenas no final)
cols_to_keep = {'Time', 'WindSpeed', 'Dir_Sin', 'Dir_Cos', 'Hour_Sin', 'Hour_Cos', 'Wind_Acc', 'Target_WindSpeed', '
    BlockID'};
data_final = data(:, cols_to_keep);

% Mostrar o resultado final
disp('Primeiras 5 linhas do dataset final pronto para modelo:');
disp(head(data_final, 5));

% Exportar os dados para CSV
writetable(data, 'windSpeed_Processed.csv');
writetable(data_final, 'windSpeed_ModelReady.csv');
fprintf('\nDatasets exportados com sucesso!\n');
fprintf('-> windSpeed_Processed.csv (Com features originais para EDA)\n');
fprintf('-> windSpeed_ModelReady.csv (Apenas features compactadas para o Modelo)\n');
```

Listing 10: windspeed_data_visualization.m

```
%% Projeto 2: Redes Neuronais Artificiais - Wind Speed
% Script: Visualização Exploratória de Dados (EDA)
clc; clearvars; close all;

%% 1. Carregamento dos Dados Processados
path = "windSpeed_Processed.csv";

if ~isfile(path)
    error('Ficheiro não encontrado! Execute primeiro o script data_cleaning.m para gerar o CSV.');
end

data = readtable(path);

% Garantir que a coluna de Tempo é lida como datetime para os plots temporais
if iscell(data.Time) || isstring(data.Time)
    data.Time = datetime(data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
end

fprintf('=============================================\n');
fprintf('        ANÁLISE EXPLORATÓRIA (EDA)        \n');
fprintf('=============================================\n');
```

```
%% 2. Visualização de Dados e Continuidade
fprintf('Gerando visualizações estatísticas...\n');

gaps_longos = sum(data.TimeDiff >= 60);
fprintf('Contagem de intervalos de tempo >= 60 segundos: %d\n', gaps_longos);

% Figura 1: Distribuição das Features
figure('Name', 'Features Distribution', 'NumberTitle', 'off', 'Position', [100, 100, 800, 900]);
subplot(3,1,1);
histogram(data.WindSpeed, 'NumBins', 50, 'FaceColor', [0 0.4470 0.7410], 'EdgeColor', 'w');
set(gca, 'YScale', 'log');
title('Wind Speed Distribution (Log Scale)');
xlabel('Wind Speed (log(1 + m/s))'); ylabel('Frequency (Log)'); grid on;

subplot(3,1,2);
histogram(data.WindDirection, 'FaceColor', [0.8500 0.3250 0.0980], 'EdgeColor', 'w');
title('Wind Direction Distribution');
xlabel('Direction (Degrees)'); ylabel('Frequency'); grid on;

subplot(3,1,3);
histogram(data.Wind_Acc, 'NumBins', 50, 'FaceColor', [0.4940 0.1840 0.5560], 'EdgeColor', 'w');
set(gca, 'YScale', 'log');
title('Wind Acceleration (\Delta v) Distribution (Log Scale)');
xlabel('Acceleration (m/s^2)'); ylabel('Frequency (Log)'); grid on;

% Figura 2: Análise de Gaps Temporais
figure('Name', 'Time Gaps Analysis', 'NumberTitle', 'off', 'Position', [150, 150, 800, 600]);

% Subplot 1: Histograma dos intervalos normais (Zoom na amostragem)
subplot(2,1,1);
normalGaps = data.TimeDiff(data.TimeDiff < 60);
histogram(normalGaps, 'BinWidth', 1, 'FaceColor', [0.4660 0.6740 0.1880], 'EdgeColor', 'w');
set(gca, 'YScale', 'log');
title('Time Intervals Histogram (Normal Sampling < 60s) - Log Scale');
xlabel('Seconds between Samples'); ylabel('Count (Log)'); grid on;

% Subplot 2: Ocorrência de Gaps ao longo do tempo (Revertido para pontos)
subplot(2,1,2);
plot(data.Time, data.TimeDiff, '.', 'MarkerSize', 8, 'Color', [0.6350 0.0780 0.1840]);
title('Gaps Occurrence over Timeline');
xlabel('Time'); ylabel('Gap Duration (seconds)');
grid on;

%% 3. Análise de Features (Correlação e Dispersão)
fprintf('Analisando correlações entre features e o Target...\n');

% Figura 3: Matriz de Correlação Expandida
figure('Name', 'Correlation Matrix', 'NumberTitle', 'off', 'Position', [200, 200, 800, 650]);
vars_corr = {'WindSpeed', 'Dir_Sin', 'Dir_Cos', 'Hour_Sin', 'Hour_Cos', 'Wind_Acc', 'Target_WindSpeed'};
vars_display = {'Wind Speed', 'Dir Sin', 'Dir Cos', 'Hour Sin', 'Hour Cos', 'Wind Acc', 'Target Speed'};
corr_matrix = corr(table2array(data(:, vars_corr)), 'Rows', 'complete');
heatmap(vars_display, vars_display, corr_matrix, 'Title', 'Correlation Matrix');

% Figura 4: Dispersões (Scatter Plots) vs Target
figure('Name', 'Scatter Plots vs Target', 'NumberTitle', 'off', 'Position', [250, 250, 1200, 800]);
scatter_vars = {'WindSpeed', 'Dir_Sin', 'Dir_Cos', 'Hour_Sin', 'Hour_Cos', 'Wind_Acc'};
titles = {'Current Wind Speed', 'Direction Sine', 'Direction Cosine', 'Hour Sine', 'Hour Cosine', 'Wind Acceleration'};
x_labels = {'Wind Speed (log(1 + m/s))', 'Sin(\theta)', 'Cos(\theta)', 'Sin(Hour)', 'Cos(Hour)', 'Acceleration (m/s^2)'};

for i = 1:6
    subplot(2, 3, i);
    scatter(data.(scatter_vars{i}), data.Target_WindSpeed, 10, 'filled', 'MarkerFaceAlpha', 0.2);
    title([titles{i} ' vs Target (+60s)']);
    xlabel(x_labels{i}); ylabel('Target (+60s) (log(1 + m/s))'); grid on;
end

%% 4. Evolução Temporal de Ambas as Variáveis
fprintf('Gerando gráficos de evolução temporal...\n');
figure('Name', 'Temporal Evolution', 'NumberTitle', 'off', 'Position', [300, 100, 1000, 800]);

subplot(3,1,1);
plot(data.Time, data.WindSpeed, 'Color', [0 0.4470 0.7410]);
title('Temporal Evolution of Wind Speed');
xlabel('Time'); ylabel('Wind Speed (log(1 + m/s))'); grid on;

subplot(3,1,2);
plot(data.Time, data.WindDirection, '.', 'Color', [0.8500 0.3250 0.0980], 'MarkerSize', 2);
title('Temporal Evolution of Wind Direction');
xlabel('Time'); ylabel('Wind Direction (Degrees)'); grid on;

subplot(3,1,3);
plot(data.Time, data.Wind_Acc, 'Color', [0.4940 0.1840 0.5560]);
```

```
title('Abrupt Changes (Wind Acceleration \Delta v over Time)');
xlabel('Time'); ylabel('Acceleration (m/s^2)'); grid on;

%% 5. Comportamento Diário, Mensal e Outliers
fprintf('Gerando gráficos de comportamento diário, mensal e outliers...\n');
figure('Name', 'Boxplots: Outliers, Daily and Monthly', 'NumberTitle', 'off', 'Position', [350, 150, 1200, 450]);

subplot(1,3,1);
boxplot(data.WindSpeed);
title('Outliers Identification (Overall)');
ylabel('Wind Speed (log(1 + m/s))'); grid on;

subplot(1,3,2);
boxplot(data.WindSpeed, hour(data.Time));
title('Daily Behavior (Wind Speed Distribution per Hour)');
xlabel('Hour of the Day (0-23)'); ylabel('Wind Speed (log(1 + m/s))'); grid on;

subplot(1,3,3);
boxplot(data.WindSpeed, month(data.Time));
title('Monthly Behavior (Wind Speed Distribution per Month)');
xlabel('Month of the Year (1-12)'); ylabel('Wind Speed (log(1 + m/s))'); grid on;

%% 6. Relação aos Pares Adicional (Wind Speed vs Direction)
fprintf('Gerando relação aos pares extra (Speed vs Direction)...\n');
figure('Name', 'Pairwise: Speed vs Direction', 'NumberTitle', 'off', 'Position', [500, 300, 700, 500]);
scatter(data.WindDirection, data.WindSpeed, 5, 'filled', 'MarkerFaceAlpha', 0.1, 'MarkerFaceColor', [0.25 0.25 0.25]);
title('Pairwise Relationship: Wind Speed vs Wind Direction');
xlabel('Wind Direction (Degrees)'); ylabel('Wind Speed (log(1 + m/s))');
grid on;
fprintf('==============================================\n');
```

Listing 11: windspeed_model_training.m

```
%% Projeto 2: Redes Neuronais Artificiais - Wind Speed
% Script: Treino e Otimização de Hiperparâmetros (Grid Search + Teorema Limite Central)
clc; clearvars; close all;

%% 1. Carregamento dos Dados
path = "windSpeed_ModelReady.csv";

if ~isfile(path)
    error('Ficheiro não encontrado! Execute primeiro o script data_cleaning.m');
end

data = readtable(path);
inputs = table2array(data(:, {'WindSpeed', 'Dir_Sin', 'Dir_Cos', 'Hour_Sin', 'Hour_Cos', 'Wind_Acc'}))';
targets = data.Target_WindSpeed';

%% 2. Partição Cronológica Rigorosa (Prevenção de Data Leakage)
% A divisão é feita rigorosamente ao início para isolar os dados de Teste.
% Guardamos os últimos 10% da série temporal no "cofre".
total_samples = length(targets);
split_idx = floor(0.90 * total_samples);

inputs_tv = inputs(:, 1:split_idx);
targets_tv = targets(1:split_idx);

inputs_test = inputs(:, split_idx+1:end);
targets_test = targets(split_idx+1:end);

fprintf('==============================================\n');
fprintf('        PREPARAÇÃO DOS DADOS (SPLIT)        \n');
fprintf('==============================================\n');
fprintf('Treino/Validação (Optimização): %d amostras\n', split_idx);
fprintf('Teste Final (Cofre isolado):    %d amostras\n\n', total_samples - split_idx);

%% 3. Configuração de Hardware
fprintf('Algoritmo de treino definido para Levenberg-Marquardt (trainlm).\n\n');
use_gpu_flag = 'no';
train_algo = 'trainlm';

%% 4. Definição do Espaço de Otimização
% Parâmetros: 1) Número de Neurónios, 2) Função de Ativação

% Teorema do Limite Central dita N >= 30 para assumir normalidade
num_runs = 30;

% Construção Automática da Grelha (Grid Search)
neurons_list = [5, 10, 15, 20];
act_list = {'tansig', 'poslin'};

idx = 1;
```

```
opt_space = struct('neurons', {}, 'activation', {});
for n = 1:length(neurons_list)
    for a = 1:length(act_list)
        opt_space(idx).neurons = neurons_list(n);
        opt_space(idx).activation = act_list{a};
        idx = idx + 1;
    end
end
num_points = length(opt_space); % 4 x 2 = 8 Pontos

mean_mse_val = zeros(num_points, 1);
var_mse_val = zeros(num_points, 1);

fprintf('==============================================\n');
fprintf('    OTIMIZAÇÃO DE REDE NEURONAL (30 RUNS)    \n');
fprintf('==============================================\n');
fprintf('Total de redes a treinar: %d treinos\n\n', num_points * num_runs);

%% 5. Ciclo de Otimização
for p = 1:num_points
    n_neurons = opt_space(p).neurons;
    act_func = opt_space(p).activation;

    fprintf('A avaliar Ponto %d/%d [Neurónios: %2d | Ativação: %6s]:\n', p, num_points, n_neurons, act_func);

    val_mse_runs = zeros(num_runs, 1);

    for r = 1:num_runs
        fprintf('  -> Treinando modelo %2d/%d do Ponto %d ... ', r, num_runs, p);

        % Configurar a rede feedforward com o algoritmo compatível
        net = fitnet(n_neurons, train_algo);
        net.layers{1}.transferFcn = act_func;

        % Fixar a paciência num valor estável por defeito para todas as redes
        net.trainParam.max_fail = 10;

        % ESCONDER A JANELA DE TREINO (Muito importante para loops longos!)
        net.trainParam.showWindow = false;

        % Treinar com MSE para penalizar picos de erro grandes
        net.performFcn = 'mse';

        % Configuração para Séries Temporais (Divisão Sequencial em vez de Aleatória)
        net.divideFcn = 'divideblock';

        % Divisão feita APENAS nos dados de TV (O Teste real é 0 aqui!)
        net.divideParam.trainRatio = 0.85;
        net.divideParam.valRatio   = 0.15;
        net.divideParam.testRatio  = 0.00;

        % Treinar a rede
        [net, tr] = train(net, inputs_tv, targets_tv, 'useGPU', use_gpu_flag);

        % Extrair a métrica de Validação para avaliar quão boa foi esta execução
        val_mse_runs(r) = tr.best_vperf;

        fprintf('Concluído! (Val MSE: %.4f)\n', val_mse_runs(r));
    end

    mean_mse_val(p) = mean(val_mse_runs);
    var_mse_val(p) = var(val_mse_runs);

    fprintf('=> RESUMO PONTO %d | Média Val MSE: %.4f | Variância: %.6f\n\n', p, mean_mse_val(p), var_mse_val(p));
end

%% 6. Tabela de Resultados Finais
fprintf('\n==============================================\n');
fprintf('        RESULTADOS DA OTIMIZAÇÃO            \n');
fprintf('==============================================\n');

neur_col = [opt_space.neurons]';
act_col = string({opt_space.activation}');

T = table(neur_col, act_col, mean_mse_val, var_mse_val, ...
    'VariableNames', {'Neuronios', 'Ativacao', 'Media_Val_MSE', 'Variancia_Val_MSE'});
disp(T);

% Gerar Gráfico de Barras com Error Bars para o Relatório
fprintf('A gerar gráfico de comparação (Error Bar Chart)...\n');
config_labels = strcat(string(neur_col), " neur. (", act_col, ")");

figure('Name', 'Optimization Performance', 'NumberTitle', 'off', 'Position', [200, 200, 900, 500]);
```

```
% Adicionar Error Bars com os pontos da média (sem as barras de histograma)
std_mse_val = sqrt(var_mse_val);
errorbar(1:num_points, mean_mse_val, std_mse_val, 'k', 'LineStyle', 'none', 'Marker', '.', 'MarkerSize', 25, 'LineWidth', 1.5, 'CapSize', 12);
hold on;

xticks(1:num_points);
xticklabels(config_labels);
xtickangle(25);
xlabel('Configuração do Modelo');
ylabel('Mean Validation MSE (Erro)');
title('Comparação de Hiperparâmetros (Média e Desvio Padrão em 30 Execuções)');
grid on;
hold off;

%% 7. Treino do Modelo Final e Avaliação com MAE
% Descobrir o melhor ponto da tabela
[~, best_idx] = min(mean_mse_val);
best_config = opt_space(best_idx);

fprintf('\n==============================================\n');
fprintf('        TREINO DO MODELO FINAL (BEST)       \n');
fprintf('==============================================\n');
fprintf('Melhor Ponto: %d (Neurónios: %d, Ativação: %s)\n', ...
    best_idx, best_config.neurons, best_config.activation);
fprintf('A treinar modelo definitivo e a abrir janela (Show Window)...\n');

final_net = fitnet(best_config.neurons, train_algo);
final_net.layers{1}.transferFcn = best_config.activation;
final_net.trainParam.max_fail = 10;
final_net.performFcn = 'mse';
final_net.divideFcn = 'divideblock';
final_net.divideParam.trainRatio = 0.85;
final_net.divideParam.valRatio   = 0.15;
final_net.divideParam.testRatio  = 0.00;

% ATIVAR JANELA para captura dos gráficos (Curvas de Treino, Error Histogram, etc)
final_net.trainParam.showWindow = true;

% Treinar na fatia de otimização inteira
[final_net, final_tr] = train(final_net, inputs_tv, targets_tv, 'useGPU', use_gpu_flag);

%% 8. Avaliação Cega no Cofre (Test Set)
fprintf('\n==============================================\n');
fprintf('         AVALIAÇÃO FINAL (TEST SET)         \n');
fprintf('==============================================\n');

% Extrair previsões dos dados que a rede NUNCA viu
final_preds = final_net(inputs_test);

% Reverter a transformação logarítmica (expm1) para voltar a m/s
final_preds_ms = expm1(final_preds);
targets_test_ms = expm1(targets_test);

% Calcular o MAE puramente no Test Set em unidades originais
final_mae = mae(targets_test_ms - final_preds_ms);

fprintf('Métrica Final para o Relatório:\n');
fprintf('=> MAE (Mean Absolute Error) Cego: %.4f m/s\n', final_mae);
fprintf('==============================================\n');
```

Listing 12: weatherStation_model_training.m

```
%% PROJETO 2: Redes Neuronais - Apparent Temperature
% Script: Visualização Exploratória de Dados (EDA)
clc; clearvars; close all;

%% 1 - Importação e Analise Inicial

data = readtable('weatherHistory.csv');

fprintf('--- Exploração Inicial ---\n');
fprintf('Observações: %d | Features: %d\n', size(data, 1), size(data, 2));
head(data)
summary(data)
disp('Valores em falta (NaN) por feature:');
disp(array2table(sum(ismissing(data)), 'VariableNames', data.Properties.VariableNames));

% contagem de zeros

zeros_count = zeros(1, size(data, 2));
for i = 1:size(data, 2)
```

```
    col_data = data{:, i};
    if isnumeric(col_data)
        zeros_count(i) = sum(col_data == 0, "omitnan");
        else
        % Contabilizar de forma segura mesmo em colunas lidas como string/texto
        num_data = str2double(string(col_data));
        zeros_count(i) = sum(num_data == 0, 'omitnan');
    end
end
disp('Contagem de valores iguais a zero:');
disp(array2table(zeros_count, 'VariableNames', data.Properties.VariableNames));

%% 2 - Exploração e Analise dos Dados (EDA)
fprintf('---------------------------------\n');
fprintf('Visualizações Estatisticas\n');
fprintf('---------------------------------\n');

features = data.Properties.VariableNames;
n = length(features);

% ------ GRAPH 1 - HISTOGRAMA --------------

figure('Name', 'Features Distribution', 'NumberTitle', 'off', 'Position', [100, 100, 1000, 800]);

for i = 1:n
    % Define a posição do gráfico: 3 colunas, linhas automáticas
    subplot(ceil(n/3), 3, i);

    % Extrair os dados da coluna atual
    % data.(features{i}) acede dinamicamente à coluna pelo nome
    histogram(data.(features{i}), 'FaceColor', [0.2 0.5 0.7]);

    % Título dinâmico usando o nome da feature
    title(['Distribuição de: ' features{i}], 'Interpreter', 'none');
    grid on;
end


% ------ GRAPH 2 - BOXPLOTS --------------

figure('Name', 'Boxplots: Todas as Features', 'NumberTitle', 'off', 'Position', [100, 100, 1200, 500]);
for i = 1:length(features)
    subplot(1, length(features), i);
    boxplot(data.(features{i}));
    title(features{i}, 'Interpreter', 'none', 'FontSize', 8);
    grid on;
end
sgtitle('Identificação de Outliers por Variável');

% ------ GRAPH 3 - MATRIZ DE CORRELAÇÃO --------------

figure('Name', 'Correlação Completa', 'NumberTitle', 'off', 'Position', [200, 200, 800, 600]);
corrMat = corr(table2array(data));
h = heatmap(features, features, corrMat, 'CellLabelFormat', '%.2f');
h.Title = 'Correlação: Como cada variável afeta a Apparent Temperature';
h.Colormap = jet;

% ------ GRAPH 4 - SCATTERPLOT --------------

targetVar = 'ApparentTemperature_C_';

todasVars = data.Properties.VariableNames;
inputVars = setdiff(todasVars, {targetVar, 'LoudCover'}, 'stable');

numVars = length(inputVars);
numCols = 3; % Fixamos 3 colunas para ficar legível
numRows = ceil(numVars / numCols); % Calcula as linhas necessárias (ex: 7 vars -> 3 linhas)

figure('Name', 'Relação com o Target (Apparent Temperature)', 'NumberTitle', 'off', 'Position', [100, 100, 1200, 800]);

for i = 1:numVars
    % O subplot agora é dinâmico: usa numRows em vez de 2
    subplot(numRows, numCols, i);

    % Dados originais
    xData = data.(inputVars{i});
    yData = data.(targetVar);

    % Criar scatter plot
    scatter(xData, yData, 5, 'filled', 'MarkerFaceAlpha', 0.2);

    xlabel(inputVars{i}, 'Interpreter', 'none');
    ylabel('Apparent Temp (C)');
```

```
    title(['Rel: ' inputVars{i}], 'Interpreter', 'none');
    grid on;

    % Adicionar linha de tendência (ignora NaNs para não dar erro no polyfit)
    idxValid = ~isnan(xData) & ~isnan(yData);
    if sum(idxValid) > 2
        p = polyfit(xData(idxValid), yData(idxValid), 1);
        hold on;
        plot(xData, polyval(p, xData), 'r-', 'LineWidth', 1.5);
    end
end


%% 3 - Limpeza inicial dos dados

% Com base nestes dados podemos remover a coluna de Cloud Cover pois
% apresenta demasiados zeros e nao apresenta dados que possam ser
% relevantes para a rede neuronal

%  Remover a coluna problemática (Loud Cover)
data.LoudCover = [];

%  Tratar a Pressão (Imputação em vez de Remoção)
% Substituímos zeros por NaN para usar as funções inteligentes do MATLAB
data.Pressure_millibars_(data.Pressure_millibars_ < 900) = NaN;
% Para regressão (não-série temporal), usamos a mediana em vez de interpolação linear
data.Pressure_millibars_ = fillmissing(data.Pressure_millibars_, 'constant', median(data.Pressure_millibars_, 'omitnan'));

%  Tratar a Humidade (se houver zeros isolados)
data.Humidity(data.Humidity <= 0) = NaN;
data.Humidity = fillmissing(data.Humidity, 'constant', median(data.Humidity, 'omitnan'));


%Identificar e corrigir zeros na HUMIDADE (<= 0)
% A humidade costuma estar entre 0.2 e 1.0 (20% a 100%)
idxHumidadeZero = data.Humidity <= 0;
data.Humidity(idxHumidadeZero) = NaN;
data.Humidity = fillmissing(data.Humidity, 'constant', median(data.Humidity, 'omitnan'));

%  Verificação Final: Se ainda restarem NaNs (ex: no início/fim do ficheiro)
% que a interpolação não apanhou, removemos apenas esses (serão muito poucos)
data = rmmissing(data);

%% 4 -Feature Engineering

%decomposição do Wind Bearing para Cosseno e Seno

data.Wind_Sin = sin(deg2rad(data.WindBearing_degrees_));
data.Wind_Cos = cos(deg2rad(data.WindBearing_degrees_));

data.WindBearing_degrees_ = [];

% Heat Index Proxy
% Temperatura X Humidade

data.Heat_Index_Proxy = data.Temperature_C_ .* data.Humidity;

% Wind Chill Effect
% Vento X Temperatura

data.Wind_Chill_Effect = data.WindSpeed_km_h_ .* data.Temperature_C_;

% 2. Interação Humidade e Pressão (Densidade/Estabilidade do ar)
data.Hum_Press_Ratio = data.Humidity .* data.Pressure_millibars_;

%% 5 - Separação de Features e Target

targetName = 'ApparentTemperature_C_';
inputNames = setdiff(data.Properties.VariableNames, targetName, 'stable');

X = data{:, inputNames}';
y = data{:, targetName}';

% Atualizamos a lista de features para a nova tabela
features = data.Properties.VariableNames;
n = length(features);

% graficos com features novas

% ------ GRAPH 1 - HISTOGRAMA --------------

figure('Name', 'Features Distribution com features novas', 'NumberTitle', 'off', 'Position', [100, 100, 1000, 800]);

```

```
for i = 1:n
    % Define a posição do gráfico: 3 colunas, linhas automáticas
    subplot(ceil(n/3), 3, i);

    % Extrair os dados da coluna atual
    % data.(features{i}) acede dinamicamente à coluna pelo nome
    histogram(data.(features{i}), 'FaceColor', [0.2 0.5 0.7]);

    % Título dinâmico usando o nome da feature
    title(['Distribuição de: ' features{i}], 'Interpreter', 'none');
    grid on;
end


% ------ GRAPH 2 - BOXPLOTS --------------

figure('Name', 'Boxplots: Todas as Features novas', 'NumberTitle', 'off', 'Position', [100, 100, 1200, 500]);
for i = 1:length(features)
    subplot(1, length(features), i);
    boxplot(data.(features{i}));
    title(features{i}, 'Interpreter', 'none', 'FontSize', 8);
    grid on;
end
sgtitle('Identificação de Outliers por Variável');

% ------ GRAPH 3 - MATRIZ DE CORRELAÇÃO --------------

figure('Name', 'Correlação Completa com features novas', 'NumberTitle', 'off', 'Position', [200, 200, 800, 600]);
corrMat = corr(table2array(data));
h = heatmap(features, features, corrMat, 'CellLabelFormat', '%.2f');
h.Title = 'Correlação: Como cada variável afeta a Apparent Temperature';
h.Colormap = jet;

% ------ GRAPH 4 - SCATTERPLOT --------------

targetVar = 'ApparentTemperature_C_';

todasVars = data.Properties.VariableNames;
inputVars = setdiff(todasVars, {targetVar, 'LoudCover'}, 'stable');

numVars = length(inputVars);
numCols = 3; % Fixamos 3 colunas para ficar legível
numRows = ceil(numVars / numCols); % Calcula as linhas necessárias (ex: 7 vars -> 3 linhas)

figure('Name', 'Relação com o Target (Apparent Temperature) com features novas', 'NumberTitle', 'off', 'Position', [100,
     100, 1200, 800]);

for i = 1:numVars
    % O subplot agora é dinâmico: usa numRows em vez de 2
    subplot(numRows, numCols, i);

    % Dados originais
    xData = data.(inputVars{i});
    yData = data.(targetVar);

    % Criar scatter plot
    scatter(xData, yData, 5, 'filled', 'MarkerFaceAlpha', 0.2);

    xlabel(inputVars{i}, 'Interpreter', 'none');
    ylabel('Apparent Temp (C)');
    title(['Rel: ' inputVars{i}], 'Interpreter', 'none');
    grid on;

    % Adicionar linha de tendência (ignora NaNs para não dar erro no polyfit)
    idxValid = ~isnan(xData) & ~isnan(yData);
    if sum(idxValid) > 2
        p = polyfit(xData(idxValid), yData(idxValid), 1);
        hold on;
        plot(xData, polyval(p, xData), 'r-', 'LineWidth', 1.5);
    end
end


% Removemos Wind_Sen e Cos porque a sua correlação é basicamente nula, ou
% seja é só ruido.
data_final = removevars(data, {'Wind_Sin', 'Wind_Cos'});

inputNames = setdiff(data_final.Properties.VariableNames, targetName, 'stable');

%% 6 - Partição de Dados e Configuração de Hardware

X_raw = data_final{:, inputNames}';
y_raw = data_final{:, targetName}';
```

```
% Partição Aleatória Rigorosa (90% Treino/Validação | 10% Teste)
total_samples = length(y_raw);
split_idx = floor(0.90 * total_samples);

% Baralhar os índices para garantir amostragem representativa
rng(42); % Fixar semente para reprodutibilidade e isolamento do cofre
shuffled_idx = randperm(total_samples);

inputs_tv = X_raw(:, shuffled_idx(1:split_idx));
targets_tv = y_raw(shuffled_idx(1:split_idx));

inputs_test = X_raw(:, shuffled_idx(split_idx+1:end));
targets_test = y_raw(shuffled_idx(split_idx+1:end));

fprintf('==============================================\n');
fprintf('␣␣␣␣␣␣␣␣PREPARAÇÃO␣DOS␣DADOS␣(SPLIT)␣␣␣␣␣␣␣\n');
fprintf('==============================================\n');
fprintf('Treino/Validação␣(Optimização):␣%d␣amostras\n', split_idx);
fprintf('Teste␣Final␣(Cofre␣isolado):␣␣␣␣%d␣amostras\n\n', total_samples - split_idx);

use_gpu_flag = 'no';
train_algo = 'trainlm';

%% 7 - Definição do Espaço de Otimização (Grid Search)

% Teorema do Limite Central (N >= 30)
num_runs = 30;

% Construção Automática da Grelha
neurons_list = [5, 10, 15, 20];
act_list = {'tansig', 'poslin'};

idx = 1;
opt_space = struct('neurons', {}, 'activation', {});
for n = 1:length(neurons_list)
    for a = 1:length(act_list)
        opt_space(idx).neurons = neurons_list(n);
        opt_space(idx).activation = act_list{a};
        idx = idx + 1;
    end
end
num_points = length(opt_space);

mean_mse_val = zeros(num_points, 1);
var_mse_val = zeros(num_points, 1);

fprintf('==============================================\n');
fprintf('␣␣␣␣OTIMIZAÇÃO␣DE␣REDE␣NEURONAL␣(30␣RUNS)␣␣␣␣\n');
fprintf('==============================================\n');
fprintf('Total␣de␣redes␣a␣treinar:␣%d␣treinos\n\n', num_points * num_runs);

%% 8 - Ciclo de Otimização
for p = 1:num_points
    n_neurons = opt_space(p).neurons;
    act_func = opt_space(p).activation;

    fprintf('A␣avaliar␣Ponto␣%d/%d␣[Neurónios:␣%2d␣|␣Ativação:␣%6s]:\n', p, num_points, n_neurons, act_func);

    val_mse_runs = zeros(num_runs, 1);

    for r = 1:num_runs
        fprintf('␣␣->␣Treinando␣modelo␣%2d/%d␣do␣Ponto␣%d␣...␣', r, num_runs, p);

        net = fitnet(n_neurons, train_algo);
        net.layers{1}.transferFcn = act_func;

        % Normalização interna (Apenas calculada no Treino)
        net.inputs{1}.processFcns = {'removeconstantrows','mapminmax'};
        net.outputs{2}.processFcns = {'removeconstantrows','mapminmax'};

        net.trainParam.max_fail = 10;
        net.trainParam.showWindow = false; % ESCONDER JANELA
        net.performFcn = 'mse';

        % Divisão aleatória na validação cruzada interna
        net.divideFcn = 'dividerand';
        net.divideParam.trainRatio = 0.85;
        net.divideParam.valRatio   = 0.15;
        net.divideParam.testRatio  = 0.00;

        [net, tr] = train(net, inputs_tv, targets_tv, 'useGPU', use_gpu_flag);

```

```
        val_mse_runs(r) = tr.best_vperf;
        fprintf('Concluído! (Val MSE: %.4f)\n', val_mse_runs(r));
    end

    mean_mse_val(p) = mean(val_mse_runs);
    var_mse_val(p) = var(val_mse_runs);

    fprintf('=> RESUMO PONTO %d | Média Val MSE: %.4f | Variância: %.6f\n\n', p, mean_mse_val(p), var_mse_val(p));
end

%% 9 - Tabela de Resultados Finais e Gráfico
fprintf('\n==============================================\n');
fprintf('        RESULTADOS DA OTIMIZAÇÃO           \n');
fprintf('==============================================\n');

neur_col = [opt_space.neurons]';
act_col = string({opt_space.activation}');

T = table(neur_col, act_col, mean_mse_val, var_mse_val, ...
    'VariableNames', {'Neuronios', 'Ativacao', 'Media_Val_MSE', 'Variancia_Val_MSE'});
disp(T);

fprintf('A gerar gráfico de comparação (Error Bar Chart)...\n');
config_labels = strcat(string(neur_col), " neur. (", act_col, ")");

figure('Name', 'Optimization Performance', 'NumberTitle', 'off', 'Position', [200, 200, 900, 500]);
std_mse_val = sqrt(var_mse_val);
errorbar(1:num_points, mean_mse_val, std_mse_val, 'k', 'LineStyle', 'none', 'Marker', '.', 'MarkerSize', 25, 'LineWidth',
     1.5, 'CapSize', 12);
hold on;
xticks(1:num_points);
xticklabels(config_labels);
xtickangle(25);
xlabel('Configuração do Modelo');
ylabel('Mean Validation MSE (Erro)');
title('Comparação de Hiperparâmetros (Média e Desvio Padrão em 30 Execuções)');
grid on;
hold off;

%% 10 - Treino do Modelo Final (Best Configuration)
[~, best_idx] = min(mean_mse_val);
best_config = opt_space(best_idx);

fprintf('\n==============================================\n');
fprintf('        TREINO DO MODELO FINAL (BEST)       \n');
fprintf('==============================================\n');
fprintf('Melhor Ponto: %d (Neurónios: %d, Ativação: %s)\n', best_idx, best_config.neurons, best_config.activation);
fprintf('A treinar modelo definitivo e a abrir janela (Show Window)...\n');

final_net = fitnet(best_config.neurons, train_algo);
final_net.layers{1}.transferFcn = best_config.activation;
final_net.inputs{1}.processFcns = {'removeconstantrows','mapminmax'};
final_net.outputs{2}.processFcns = {'removeconstantrows','mapminmax'};
final_net.trainParam.max_fail = 10;
final_net.performFcn = 'mse';
final_net.divideFcn = 'dividerand';
final_net.divideParam.trainRatio = 0.85;
final_net.divideParam.valRatio   = 0.15;
final_net.divideParam.testRatio  = 0.00;

final_net.trainParam.showWindow = true; % ATIVAR JANELA NO ÚLTIMO MODELO

[final_net, final_tr] = train(final_net, inputs_tv, targets_tv, 'useGPU', use_gpu_flag);

%% 11 - Avaliação Cega no Cofre (Test Set com MAE)
fprintf('\n==============================================\n');
fprintf('        AVALIAÇÃO FINAL (TEST SET)          \n');
fprintf('==============================================\n');

final_preds = final_net(inputs_test);
final_mae = mae(targets_test - final_preds);

fprintf('Métrica Final para o Relatório:\n');
fprintf('=> MAE (Mean Absolute Error) Cego: %.4f\n', final_mae);
fprintf('==============================================\n');
```